

Sheep RNA-seq DESeq2 Analysis

Setup

Load required libraries and set working directory.

```
library(DESeq2)
library(ggplot2)
library(pheatmap)
library(tidyverse)
setwd("~/Documents/GitHub/ucd_bioinformaticscourse_2026/scripts/day2/Sheep_RNA_seq_for_Mery/DESeq2/")
```

Load Data

Read in the gene count matrix and sample metadata.

```
# Read in gene matrix
Gene_matrix <- read.table("~/Documents/GitHub/ucd_bioinformaticscourse_2026/scripts/day2/Sheep_RNA_seq_for_Mery/DESeq2/counts.txt")
dim(Gene_matrix)

## [1] 28524      9
#[1] 28524      9

# Read in pheno data
pheno_data <- read.table("~/Documents/GitHub/ucd_bioinformaticscourse_2026/scripts/day2/Sheep_RNA_seq_for_Mery/DESeq2/pheno_data.txt")
mutate(Day = factor(Day)) %>%
  column_to_rownames("Sample_ID") %>%
  mutate(Sample_ID = rownames(.))
dim(pheno_data)

## [1] 9 3
#[1] 9 3

levels(pheno_data$Day)

## [1] "D0" "D14" "D7"
#[1] "D0" "D14" "D7"

# Check that the sample order matches
all(rownames(pheno_data) == colnames(Gene_matrix))

## [1] TRUE
#[1] TRUE
```

DESeq2

Create DESeq2 object and filter lowly expressed genes

The count matrix and metadata are passed to `DESeqDataSetFromMatrix()`, with `~ Day` as the design formula, meaning the model will test for differential expression across time points (D0, D7, D14).

Lowly expressed genes are then removed: only genes with at least 10 counts in a minimum of 3 samples (the smallest group size) are retained. This reduces noise and improves statistical power.

```
# Prep data for DESeq2
# DESeqDataSetFromMatrix requires a plain matrix, so convert
dds <- DESeqDataSetFromMatrix(Gene_matrix, pheno_data, design = ~ Day)

# Filter genes
# Keep genes that have at least 10 counts across the smallest group size
smallestGroupSize <- 3
keep <- rowSums(counts(dds) >= 10) >= smallestGroupSize
dds <- dds[keep,]
dim(dds)
```

```
## [1] 15922      9
```

```
#[1] 15922      9
```

Run DESeq2

`DESeq()` runs the full differential expression pipeline in a single call. Internally, it performs three key steps:

1. **Estimation of size factors** — accounts for differences in sequencing depth between samples. Each sample gets a size factor that scales its counts so that comparisons across samples are fair. This is DESeq2's normalisation step, using the *median-of-ratios* method: for each gene, the ratio of its count to its geometric mean across all samples is calculated, and the median of these ratios becomes the sample's size factor.
2. **Estimation of dispersion** — RNA-seq count data is modelled using a negative binomial distribution, which requires knowing how much counts vary across replicates for each gene (dispersion). DESeq2 first estimates a per-gene dispersion, then shrinks these estimates toward a fitted trend to improve reliability, especially for genes with few counts.
3. **Fitting the statistical model and testing** — a generalised linear model (GLM) is fitted per gene using the design formula (`~ Day`). Wald tests are used to assess whether expression differs significantly between groups.

```
# Run DESeq2
dds <- DESeq(dds)
```

PCA

Variance-stabilising transformation

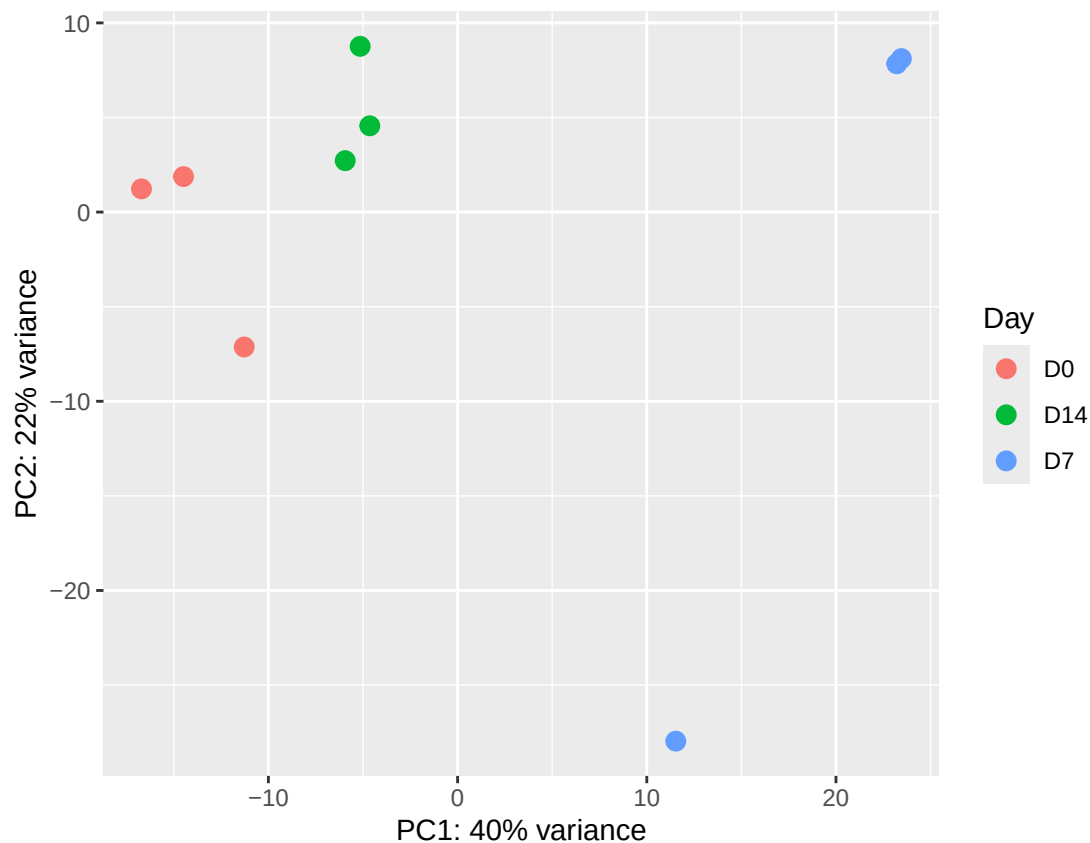
Before PCA, raw counts are transformed using `vst()` (variance-stabilising transformation). Raw counts have a mean-variance relationship — highly expressed genes naturally show more variability, which would dominate a PCA. VST corrects for this, making the variance approximately independent of the mean. `blind=FALSE` means the transformation takes the experimental design into account, which is appropriate when samples are expected to differ.

```
# Variance-stabilizing transformation
vsd <- vst(dds, blind=FALSE)
pcaData <- plotPCA(vsd, intgroup="Day", returnData=TRUE)
```

```
percentVar <- round(100 * attr(pcaData, "percentVar"))

# create plot and save to object
pcaPlot <- ggplot(pcaData, aes(x=PC1, y=PC2, color=Day)) +
  geom_point(size=3) +
  xlab(paste0("PC1: ",percentVar[1],"% variance")) +
  ylab(paste0("PC2: ",percentVar[2],"% variance")) +
  coord_fixed()

# view
pcaPlot
```



```
# save
ggsave("PCA_plot.pdf", pcaPlot, device="pdf")
```

Differential Expression Results

Contrasts are extracted using `results()`, comparing each timepoint to D0 (the reference). Genes are considered differentially expressed at adjusted p-value < 0.05 (Benjamini-Hochberg correction).

```
# Extract results for each contrast
res_D7_vs_D0 <- results(dds, contrast = c("Day", "D7", "D0"))
res_D14_vs_D0 <- results(dds, contrast = c("Day", "D14", "D0"))

# Summary of DEGs
summary(res_D7_vs_D0)
```

```
##
```

```
## out of 15922 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)      : 2839, 18%
## LFC < 0 (down)    : 2598, 16%
## outliers [1]      : 49, 0.31%
## low counts [2]    : 0, 0%
## (mean count < 4)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

```
summary(res_D14_vs_D0)
```

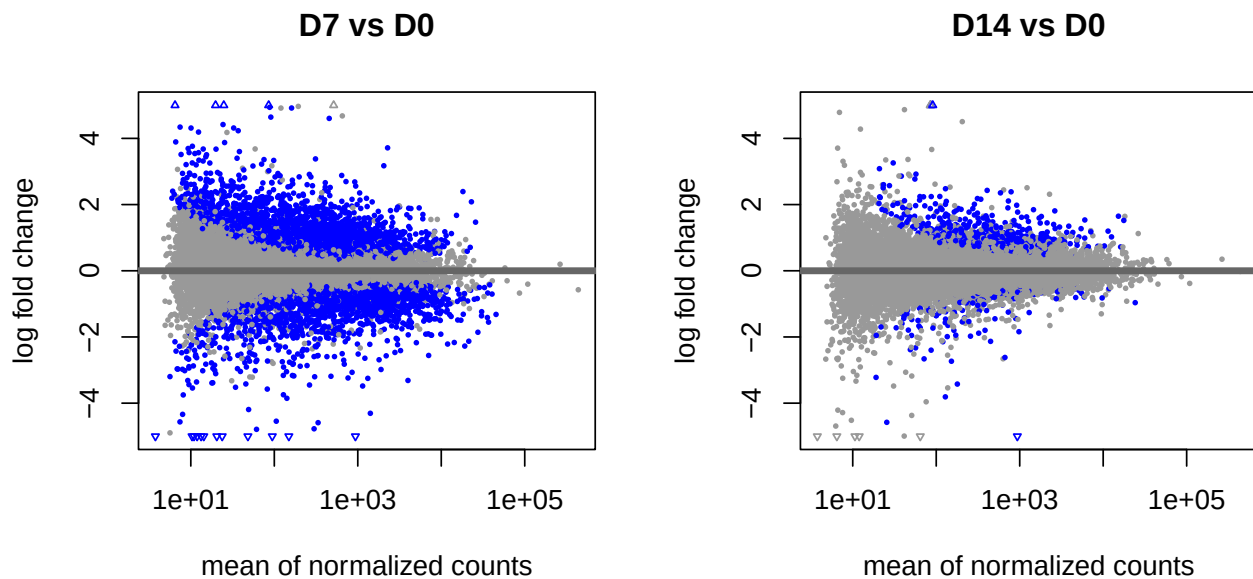
```
##
## out of 15922 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)      : 554, 3.5%
## LFC < 0 (down)    : 146, 0.92%
## outliers [1]      : 49, 0.31%
## low counts [2]    : 1540, 9.7%
## (mean count < 18)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

MA Plots

MA plots show the log2 fold change (M) against the mean normalised count (A) for every gene. Each dot is a gene; those passing the adjusted p-value threshold of 0.05 are highlighted in blue. Genes with very low counts tend to show high variability in fold change, visible as the wide spread on the left side of the plot. DESeq2's `plotMA()` also applies log fold change shrinkage by default, pulling unreliable estimates from low-count genes toward zero.

```
par(mfrow = c(1, 2))

plotMA(res_D7_vs_D0, main = "D7 vs D0", ylim = c(-5, 5))
plotMA(res_D14_vs_D0, main = "D14 vs D0", ylim = c(-5, 5))
```



```
par(mfrow = c(1, 1))
```

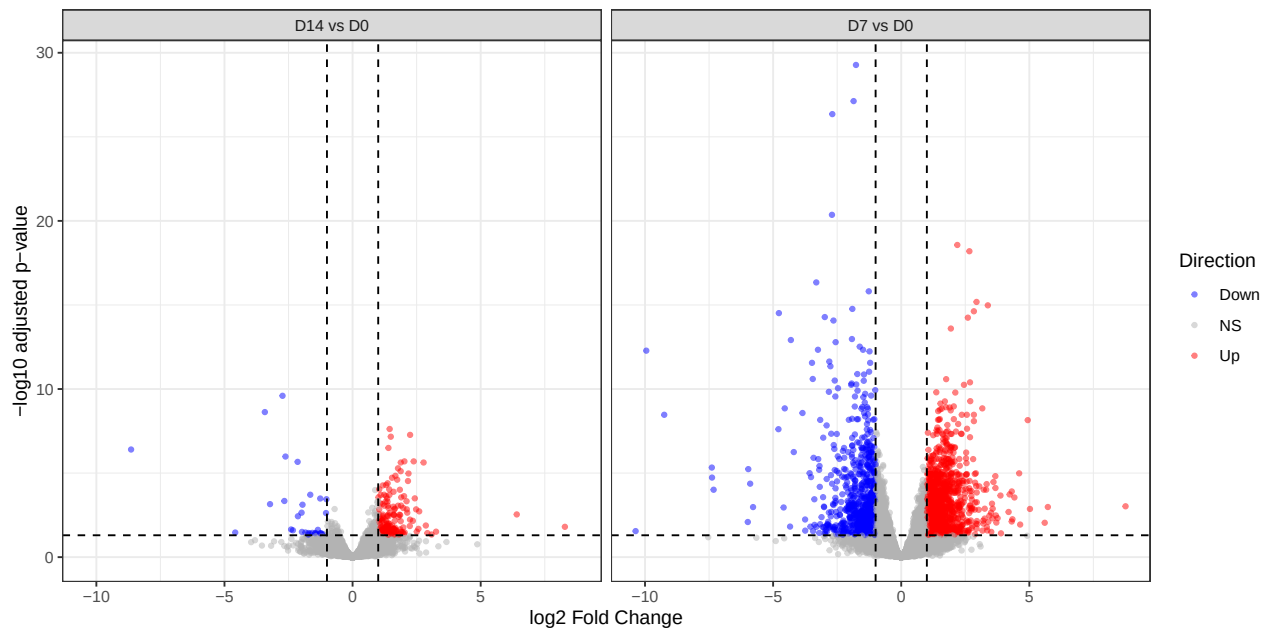
Volcano Plots

Volcano plots display log2 fold change on the x-axis against -log10 adjusted p-value on the y-axis. Genes are coloured by significance and direction: red for significantly upregulated ($\log_2\text{FC} > 1$, $\text{padj} < 0.05$), blue for significantly downregulated ($\log_2\text{FC} < -1$, $\text{padj} < 0.05$), and grey for non-significant genes. The further a gene is from the centre, the larger its fold change; the higher it sits, the more statistically significant it is.

```
# Helper function to prepare results for volcano plot
prep_volcano <- function(res, contrast_label) {
  as.data.frame(res) %>%
    rownames_to_column("gene") %>%
    filter(!is.na(padj)) %>%
    mutate(
      contrast = contrast_label,
      sig = case_when(
        padj < 0.05 & log2FoldChange > 1 ~ "Up",
        padj < 0.05 & log2FoldChange < -1 ~ "Down",
        TRUE ~ "NS"
      )
    )
}

volcano_data <- bind_rows(
  prep_volcano(res_D7_vs_D0, "D7 vs D0"),
  prep_volcano(res_D14_vs_D0, "D14 vs D0")
)

ggplot(volcano_data, aes(x = log2FoldChange, y = -log10(padj), color = sig)) +
  geom_point(alpha = 0.5, size = 1) +
  scale_color_manual(values = c("Up" = "red", "Down" = "blue", "NS" = "grey70")) +
  geom_vline(xintercept = c(-1, 1), linetype = "dashed", color = "black") +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed", color = "black") +
  facet_wrap(~ contrast) +
  labs(x = "log2 Fold Change", y = "-log10 adjusted p-value", color = "Direction") +
  theme_bw()
```



Heatmap of Top DEGs

The heatmap shows the top 50 differentially expressed genes (ranked by adjusted p-value) pooled across both contrasts. VST-normalised expression values are scaled by row (z-score per gene) so that expression patterns across samples are comparable regardless of absolute expression level. Samples are annotated by timepoint. Hierarchical clustering is applied to both rows (genes) and columns (samples), grouping genes with similar expression patterns and revealing whether samples cluster by Day.

```
# Get top 50 DEGs by padj pooled across both contrasts
top_genes <- bind_rows(
  as.data.frame(res_D7_vs_D0) %>% rownames_to_column("gene"),
  as.data.frame(res_D14_vs_D0) %>% rownames_to_column("gene")
) %>%
  filter(!is.na(padj)) %>%
  arrange(padj) %>%
  distinct(gene, .keep_all = TRUE) %>%
  slice_head(n = 50) %>%
  pull(gene)

# Extract VST counts for top genes
mat <- assay(vsd)[top_genes, ]

# Scale by row (z-score)
mat_scaled <- t(scale(t(mat)))

# Annotation for columns
annotation_col <- data.frame(Day = pheno_data$Day)
rownames(annotation_col) <- rownames(pheno_data)

pheatmap(
  mat_scaled,
  annotation_col = annotation_col,
  cluster_rows = TRUE,
  cluster_cols = TRUE,
```

```
show_rownames = TRUE,  
show_colnames = TRUE,  
fontsize_row = 7,  
main          = "Top 50 DEGs (scaled VST counts)"  
)
```

